

Plan de migración de base de datos y datos

GuramaOnline

Motor	Ventajas	Desventajas
MySQL	• Muy usado a nivel mundial, fácil de encontrar ayuda • Rápido en operaciones simples de lectura • Hosting gratis o barato disponible en casi todos lados • Buen soporte con NestJS y Prisma • Configuración inicial sencilla	• Cumple ACID pero con algunas limitaciones en transacciones complejas • Manejo de concurrencia más débil cuando hay muchas escrituras a la vez • Soporte de JSON menos maduro que otros motores • Algunas funciones avanzadas solo en versiones de pago • Menos flexible para proyectos que crecen mucho
PostgreSQL	• Cumple ACID de forma completa (más seguro para pedidos/transacciones) • Soporta JSON/JSONB, arrays y tipos personalizados • Trabaja rápido aun con muchas personas usándolo al mismo tiempo, sin bloquearse. • Soporte de primera clase en Prisma (su ORM actual) • Gratuito y con gran comunidad	• Configuración inicial un poco más compleja • Consumo de recursos ligeramente mayor • Curva de aprendizaje un poco más alta al venir de MySQL
MariaDB (motor actual)	• Totalmente gratuito, sin versiones de pago • Compatible casi al 100% con MySQL (migración mínima) • Buen soporte de la comunidad • Ligeras mejoras de rendimiento sobre MySQL • Configuración muy similar a la que ya conoces	• No resuelve el problema real de escalabilidad • Misma debilidad de concurrencia que MySQL en el fondo • Comunidad más pequeña que MySQL • No trae funciones realmente nuevas o distintas • Cambio superficial más que una mejora real

Conclusión: Elegimos el motor PostgreSQL, dado que nuestro proyecto usa actualmente MariaDB 10.4.32 (motor compatible con MySQL), y maneja muchos datos relacionales, por

lo que requerimos un motor en el que la integridad referencial no se pierda o corrompa. PostgreSQL cumple con el estándar ACID, que garantiza transacciones seguras, además tiene soporte nativo con Prisma, el ORM que usamos en el backend, lo que facilita la migración. A nuestro parecer, PostgreSQL cuenta con mayor robustez en comparación con los otros motores y encaja con lo que necesitamos para nuestro proyecto.

1. Objetivo

Migrar la base de datos de GuramaOnline desde el motor actual MariaDB 10.4.32 (compatible con MySQL, administrado vía phpMyAdmin) hacia PostgreSQL, con el fin de mejorar la escalabilidad y el rendimiento de la aplicación (especialmente en el manejo de escrituras concurrentes en los módulos de pedidos, pedidos personalizados e inventario), sin perder ni alterar la información existente durante el proceso.

2. Alcance

- **Base de datos involucrada:** base de datos de producción/desarrollo del proyecto GuramaOnline.
- **Tablas involucradas:** las 21 tablas actuales de la base de datos (gestionadas por Prisma):

1. rol_usuario	8. tipo_movimiento	15. pedido
2. tipo_documento	9. tipo_pedido	16. detalles_pedido
3. usuario	10. producto	17. ticket_compra
4. categoria	11. material	18. pedido_personalizado
5. clasificacion	12. material_color	19. detalle_pedido_personalizado
6. estado_pago	13. material_diseno	20. movimiento_material
7. metodo_pago	14. movimiento	21. notificacion

- **Fuera de alcance:** cambios en la lógica de negocio de la aplicación; solo se contempla el cambio de motor de base de datos y la migración de los datos existentes.

3. Situación actual

Motor: MariaDB versión 10.4.32 Administrado vía phpMyAdmin

ORM: Prisma versión 6.19.2

Node.js: v24.11.0

Charset/collation: utf8mb4 / utf8mb4_unicode_ci

Motor de tablas: InnoDB

Cantidad de tablas: 21

- rol_usuario
- tipo_documento
- usuario
- categoria
- clasificacion
- estado_pago
- metodo_pago
- tipo_movimiento
- tipo_pedido
- producto
- material
- material_color
- material_diseño
- movimiento
- pedido
- detalles_pedido
- ticket_compra
- pedido_personalizado
- detalle_pedido_personalizado
- movimiento_material
- notificacion

Cantidad aproximada de registros:

Número de filas (registros) con los que cuenta cada tabla en este momento.

Tabla	Registro aproximado
categoria	4
clasificacion	5
detalles_pedido	54
detalles_pedido_personalizado	55
estado_pago	4
material	3
material_color	6
material_diseño	5
metodo_pago	5
movimiento	61
movimiento_material	52
notificacion	149
pedido	87
pedido_personalizado	38
producto	28

rol_usuario	3
ticket_compra	2
tipo_documento	2
tipo_movimiento	26
tipo_pedido	2
usuario	26
Total aproximado	679 Registros

Relaciones principales:

- **usuario** se relaciona con **pedido**, **movimiento**, **movimiento_material** y **notificacion**
- **pedido** se relaciona con **detalles_pedido**, **ticket_compra** y **pedido_personalizado**
- **producto** depende de **categoria** y **clasificacion**
- **material** se relaciona con **material_color**, **material_diseno** y **movimiento_material**
- **pedido_personalizado** se relaciona con **detalle_pedido_personalizado**.

4. Situación destino

- **Motor:** PostgreSQL
- **Versión propuesta:** PostgreSQL 18
- **Servidor/ambiente:** Render (Web servise, backend dockerizado en el puerto 3000)
- **ORM:** Prisma
- **Herramienta de administración visual:** pgAdmin o Adminer (reemplazan a phpMyAdmin)

5. Inventario de datos

Datos que se migrarán, agrupados por dominio funcional del proyecto:

Dominio	Tablas incluidas
Usuarios y seguridad	usuarios, rol_usuario, tipo_documento
Catalogo/clasificación	categoria, clasificacion, producto
Materiales y variantes	material, material_color, material_diseno
Pedidos estándar	pedido, detalles_pedido, tipo_pedido
Pedidos personalizados	pedido_personalizado, detalle_pedido_personalizado
Pagos y compras	metodo_pago, estado_pago, ticket_compra
Inventario/Movimiento	movimiento, movimiento_material, tipo_movimiento
Notificación	notificacion

6. Estrategia de migración

Tipo de migración: esta migración es de tipo motor a motor (MariaDB → PostgreSQL) asistida por ORM: el esquema se traslada mediante Prisma (cambio de provider y prisma migrate deploy), y los datos se migran con un script propio que usa Prisma para escribir en el destino. No corresponde a una migración motor a servicio, ya que el destino sigue siendo un motor de base de datos relacional autoadministrado a través del mismo ORM, y no un servicio gestionado de terceros.

Se propone el siguiente enfoque, siendo Prisma el ORM ya usado en el proyecto:

1. Actualizar el esquema: modificar schema.prisma, cambiando el provider de mysql a postgresql, y ajustando los tipos ENUM y booleanos.
2. Crear la nueva base de datos vacía en PostgreSQL.
3. Aplicar las migraciones de Prisma (prisma migrate deploy) sobre PostgreSQL para crear la estructura (tablas, relaciones, índices, tipos ENUM).
4. Migrar los datos: dado que el origen es MariaDB y el tamaño de la base es pequeño, se optó por un script propio en TypeScript (migrar-datos.ts), ejecutado con ts-node, que usa la librería mysql2/promise para leer cada tabla desde MariaDB e insertar los registros equivalentes en PostgreSQL a través de Prisma, respetando el orden de dependencias entre tablas. Se descartó pgloader como alternativa porque el script propio permite controlar directamente la transformación de tipos (ENUM, booleanos) requerida por el schema de Prisma.
5. Importar los datos respetando el orden de dependencias.
6. Verificar y validar la integridad de los datos migrados.
7. Apuntar la aplicación (variable DATABASE_URL) hacia la nueva base de datos PostgreSQL.

7. Mapeo de datos

Como se mantiene el mismo ORM (Prisma) y el mismo modelo relacional, el mapeo es prácticamente 1 a 1 entre origen y destino:

Campo Original (MySQL/MariaDB)	Campo Nuevo (PostgreSQL)	Relación
usuario	usuario	id_rol_usuario → rol_usuario, t_doc → tipo_documento
producto	producto	id_categoria → categoria, id_clasificacion → clasificacion
material_color	material_color	id_material → material
material_diseno	material_diseno	id_material → material
pedido	pedido	id_usuario → usuario, id_tipo → tipo_pedido

detalles_pedido	detalles_pedido	id_pedido → pedido, id_producto → producto
pedido_personalizado	pedido_personalizado	id_pedido → pedido
detalle_pedido_personalizado	detalle_pedido_personalizado	id_ped_personal → pedido_personalizado, id_material → material, id_color → material_color (nulo permitido), id_diseno → material_diseno (nulo permitido)
ticket_compra	ticket_compra	id_pedido → pedido, id_estado → estado_pago, id_met_pago → metodo_pago
movimiento	movimiento	id_producto → producto, id_usuario → usuario, id_m → tipo_movimiento, id_material → material (nulo permitido)
movimiento_material	movimiento_material	id_producto → producto, id_usuario → usuario, id_m → tipo_movimiento, id_material → material (nulo permitido)
notificacion	notificacion	id_usuario → usuario

Al mantener el mismo schema.prisma como fuente de verdad, los nombres de tablas y campos no cambian; el ajuste real ocurre a nivel de tipos de datos.

8. Transformaciones

Aspecto	MariaDB / MySQL	PostgreSQL	Acción requerida
Autoincrementales	AUTO_INCREMENT (ej. id_producto, id_pedido)	SERIAL / IDENTITY	Prisma lo gestiona automáticamente al migrar el schema
Llaves primarias tipo ENUM	id_estado, id_met_pago, id_m, id_tipo, t_doc son ENUM(...) usados como primary key	Tipo ENUM nativo de Postgres, también válido como PK	Caso especial: revisar que Prisma/pgloader generen el CREATE TYPE ... AS ENUM correctamente antes de crear la tabla
Booleanos	tinyint(1) (ej. estado, leida en material, material_color, producto, notificacion)	BOOLEAN	Verificar que los valores 0/1 se interpreten como false/true
Fechas	datetime, datetime(3) (ej. fecha_m,	TIMESTAMP / TIMESTAMP(3)	Compatible; verificar zona horaria si se requiere

	bloqueado_hasta)		TIMESTAMP WITH TIME ZONE
Valores por defecto de fecha	current_timestamp() (ej. movimiento_material.fecha_m, notificacion.fecha)	CURRENT_TIMESTAMP	Compatible directamente
Decimales	decimal(10,2), decimal(10,0) (montos, cantidades)	NUMERIC(10,2)	Compatible, sin cambios
Enteros grandes	bigint(20) (usuario.telefono)	BIGINT	Compatible, sin cambios

9. Orden de migración

1. **Catálogos sin dependencias:** categoria, clasificacion, rol_usuario, tipo_documento, tipo_movimiento, tipo_pedido, estado_pago, metodo_pago
2. **Usuario** (depende de rol_usuario y tipo_documento)
3. **Producto** (depende de categoria y clasificacion)
4. **Material** (sin dependencias declaradas)
5. **Material_color** y **material_diseno** (dependen de material)
6. **Pedido** (depende de usuario y tipo_pedido)
7. **Detalles_pedido** (depende de pedido y producto)
8. **Pedido_personalizado** (depende de pedido)
9. **Detalle_pedido_personalizado** (depende de pedido personalizado, material, y opcionalmente material_color/material_diseno)
10. **Ticket_compra** (depende de pedido, estado_pago, metodo_pago)
11. **Movimiento** (depende de producto, usuario, tipo_movimiento, y opcionalmente material)
12. **Movimiento_material** (depende de material, usuario, tipo_movimiento, y opcionalmente pedido_personalizado)
13. **Notificacion** (depende de usuario)

10. Pruebas

Antes de migrar:

- Verificar que la base de datos MariaDB de origen no tenga errores de integridad (registros huérfanos, FKs rotas).
- Hacer un respaldo (backup) completo de la base de datos actual.
- Probar el proceso de migración primero en un ambiente de pruebas, no directamente en producción.

Después de migrar:

- Ejecutar las pruebas unitarias, de integración y automatizadas ya existentes del proyecto contra la nueva base de datos PostgreSQL.
- Probar manualmente los flujos críticos

11. Validación

- **Cantidad de registros:** comparar el número de filas por tabla entre origen y destino contra la tabla de conteo de registros (deben coincidir exactamente).
- **Integridad referencial:** confirmar que las FKs no queden rotas (por ejemplo, un movimiento_material.id_ped_personal apuntando a un pedido personalizado inexistente).
- **Datos críticos:** revisar manualmente algunos registros de detalle_pedido_personalizado con id_color/id_diseno no nulos y confirmar que se muestren igual que antes.
- **Consultas de la aplicación:** probar los endpoints del backend NestJS contra la nueva base y confirmar que las respuestas sean idénticas.
- **Funcionamiento general:** ejecutar el taller UAT (o una versión reducida) contra el nuevo ambiente.

12. Ventana de migración

Ítem	Detalle
Fecha	02/09/2026
Hora de inicio	12:45pm
Duración estimada	Dado el tamaño pequeño de la base, se estima entre 30 minutos y 2 horas, incluyendo validación
Responsable de ejecución	Camila Alejandra Mahecha Mancera

13. Plan de contingencia / rollback

- Antes de iniciar, se debe contar con un **respaldo completo** de la base de datos MariaDB original.
- Si la migración falla o se detectan inconsistencias graves en la validación:
 1. Detener el proceso de migración.
 2. Revertir la variable DATABASE_URL de la aplicación para que vuelva a apuntar a MariaDB.
 3. Restaurar el respaldo si la base de origen fue modificada durante el intento.
 4. Documentar el error encontrado antes de reintentar.
- La base de datos MariaDB original **no se elimina** hasta que la nueva base en PostgreSQL haya sido validada completamente y esté funcionando en producción por un periodo de observación.

14. Responsables

Rol	Nombre	Responsabilidad
Ejecuta la migración	Camila Alejandra Mahecha Mancera	Correr los scripts/herramientas de migración
Verifica los resultados	Camila Alejandra Mahecha Mancera	Validar integridad y pruebas
Autoriza el cambio a producción	Camila Alejandra Mahecha Mancera	Aprobar el paso final

15. Evidencias

Documentar y anexar al finalizar la migración:

- Capturas de pantalla del conteo de registros en MariaDB (antes) y PostgreSQL (después), por tabla, comparando contra la tabla de conteo de registros.
- Log de ejecución del script **migrar-datos.ts**
- Resultados de las pruebas unitarias, integración y automatizadas ejecutadas contra la nueva base.
- Captura de la aplicación funcionando correctamente contra PostgreSQL.
- Copia del schema.prisma final usado para PostgreSQL.

[Paso a paso - Migracion a PostGre.pdf](#)

16. Control de versiones

Fecha	Nº Versión	Descripción del cambio	Elaborado por
02/09/2026	1.0	Primera versión del plan	Camila Mahecha, Evelyn Cárdenas,
10/09/2026	1.1	Se agrega el tipo de migración (motor a motor) y adjunta el paso a paso	Camila Mahecha